University of Applied Sciences Ulm
Faculty of Electrical Engineering and Information Technology



Master Thesis

submitted for the degree of

Master of Engineering

Type3 Asset Administration Shell

A concept and implementation of a proactive Asset Administration Shell (AAS) with Multi-Agent-System (MAS) integration using Belief-Desire-Intention (BDI) approach

by

Ahmed Ibrahim Almohamed, BEng
Matriculation Nr.: 6005151

Submission Date: January 5, 2026
Supervisors: Prof. Dr. Sven Völker
Prof. Dr.-Ing. Hartwig Baumgärtel

Abstract

As industries move toward Industry 4.0 (I4.0), intelligent autonomous systems capable of managing physical assets are increasingly essential. The Asset Administration Shell (AAS) has emerged as a key technology for standardizing the representation and management of digital twins. Among AAS types, the Type 3 Asset Administration Shell (Type3AAS) is distinguished by its capacity to encapsulate not only an asset's data and functions but also its proactivity. This thesis proposes an approach to implementing Type3AAS using Multi Agent system (MAS) based on the Belief-Desire-Intention (BDI) architecture. A systems engineering approach was taken to define the requirements, concept, and implementation of the proposed solution. The work establishes foundational submodels for the AAS and demonstrates their application through a production resource use case involving drill machines. The proof of concept is validated by the ability of the implemented Type3AAS to autonomously manage and optimize drill machine operations in a simulated production environment. This thesis provides groundwork for further research and development in this area.

List of Acronyms

AAS Asset Administration Shell

Type3AAS Type 3 Asset Administration Shell

BDI Belief-Desire-Intention

IIoT Industrial Internet of Things

I4.0 Industry 4.0

RAMI4.0 Reference Architectural Model Industrie 4.0

MAS Multi Agent system

DT Digital Twin

XML eXtensible Markup Language

JSON JavaScript Object Notation

RDF Resource Description Framework

API Application Programming Interface

CRUD Create, Read, Update, Delete

SYNC Synchronous

ASYNC Asynchronous

LLC Low-Level Control

HLC High-Level Control

FAAAST Fraunhofer Advanced AAS Tools for Simplified Interoperability

PLC Programmable Logic Controller

ACL Agent Communication Language

FIPA Foundation for Intelligent Physical Agents

OPC UA Open Platform Communications Unified Architecture

MQTT Message Queuing Telemetry Transport

HTTP Hypertext Transfer Protocol

REST Representational State Transfer

XMPP Extensible Messaging and Presence Protocol

JADE JAVA Agent Development Framework

JADEX JADE eXtension

SPADE Smart Python Agent Development Environment

CPS Cyber Physical Systems

IDTA Industrial Digital Twin Association

ACL Agent Communication Language

List of Figures

1.1 Reference Architectural Model Industrie 4.0 (RAMI 4.0) [1] 9

2.1 The Industry 4.0 Component: Asset and its Administration Shell [2] . 13
2.2 Asset Administration Shell Meta Model [3] 14
2.3 Submodel Meta Model [3] 15
2.4 Submodel Element Meta Model [3] 16
2.5 Data Types Meta Model [3] 19
2.6 Types of Asset Administration Shells [4] 20
2.7 Concept of Type3 AAS [5] 21
2.8 Industrial Agent Layers [6] 27
2.9 BDI Reasoning Cycle [7] 31

3.1 Use Case Diagram for the SmartDrillSystem 44

List of Tables

2.1 Submodel Element Types and Definitions [3] 17
2.2 Specification of *type2 Application Programming Interface (API)* [3] . . 20
2.3 Comparison of Asset Administration Shell frameworks 36
2.4 Comparison of Multi-Agent System and BDI frameworks 36
2.5 Summary of AAS Type 3 implementations based on MAS concepts . 37

3.1 Traceability Matrix: Contributions to Requirements 48

8.1 Use Case UC1: System Startup 58
8.2 Use Case UC3: Drilling Order 59
8.3 Use Case UC5: System Shutdown 61
8.4 System Specification Requirements 63
8.5 ResourceAAS Requirements 64
8.6 ProductionAAS Requirements 65
8.7 MAS/BDI Requirements 66
8.8 Non-Functional Requirements 67

Contents

1 Introduction **8**

2 Fundamentals and State of the Art **12**

2.1 Asset Administration Shells(AAS) 12

2.1.1 Basic concept of AAS 12

2.1.2 Submodels and data types 15

2.1.3 Types of AAS 19

2.1.4 AAS Type3 21

2.1.5 Summary 22

2.2 Multi-Agent Systems (MAS) 23

2.2.1 Agents and Their Properties 23

2.2.2 Multi-Agent Systems 24

2.2.3 Agent Architectures 25

2.2.4 Agent Communication 26

2.2.5 Agent Coordination and Cooperation 26

2.2.6 Industrial Agents 27

2.2.7 Summary 28

2.3 BDI (Belief-Desire-Intention) Approach 29

2.3.1 Foundations of the BDI Model 29

2.3.2 The BDI Reasoning Cycle 30

2.3.3 BDI for Industrial and Cyber-Physical Systems 31

2.3.4 Summary 32

2.4 Software Frameworks and Platforms 32

2.4.1 Asset Administration Shell Frameworks 32

2.4.2 Multi-Agent System Frameworks . . . 34
2.4.3 Framework selection considerations . . . 35
2.4.4 Summary . . . 36
2.5 Integration of AAS with MAS . . . 37
2.5.1 Critical Analysis of Existing Approaches . . . 39
2.5.2 Addressed Research Gap . . . 40

3 Use Cases and System Requirements 42
3.1 From Research Questions to Use Cases . . . 42
3.2 System Components . . . 43
3.3 Use Cases . . . 43
3.3.1 UC1: System Startup . . . 44
3.3.2 UC3: Drilling Order . . . 44
3.3.3 UC5: System Shutdown . . . 45
3.3.4 Included Use Cases . . . 45
3.4 System Requirements . . . 46
3.4.1 Functional Requirements . . . 46
3.4.2 Non-Functional Requirements (NFR_1 – NFR_5) . . . 47
3.5 Requirements Traceability . . . 48
3.6 Summary . . . 48

4 Concept 50
4.1 System overview . . . 50
4.2 AAS Concept . . . 51
4.3 AAS-MAS Integration Concept . . . 51
4.4 BDI Agent Concept . . . 52
4.5 Proactive AAS Concept . . . 52

5 Implementation 53
5.1 AAS implementation and repository design pattern . . . 53
5.2 Resource Agents implementation . . . 54
5.3 Production Agents implementation . . . 54

5.4 MAS communication over XMPP protocols in Asynchronous way . . 55

6 Evaluation **56**

7 Outlook **57**

8 Appendix **58**

8.1 Detailed Use Cases and Requirements 58

8.1.1 Use Case Specifications 58

8.1.2 System Specifications 63

8.1.3 ResourceAAS Specifications 63

8.1.4 ProductionAAS Specifications 65

8.1.5 MAS/BDI Specifications 66

8.1.6 Non-Functional Requirements 67

Chapter 1

Introduction

The manufacturing industry is undergoing a fundamental transformation driven by the fourth industrial revolution, commonly referred to as Industry 4.0. This transformation is characterized by Cyber Physical Systems (CPS), intelligent automation, and the convergence of technologies including mechatronic systems, controllers, on-board computation, big data, machine learning, and the Industrial Internet of Things (IIoT) [8].

Industry 4.0 holds the promise of increased flexibility in manufacturing, along with mass customization, better quality, and improved productivity [9]. Its vision encompasses ecosystems of smart factories with intelligent and autonomous shop-floor entities [10]. To provide a structured approach for implementing this vision, the Platform Industrie 4.0 working group developed the Reference Architectural Model Industrie 4.0 (Reference Architectural Model Industrie 4.0 (RAMI4.0)) [11].
RAMI4.0 is a three-dimensional map showing how to approach the deployment of Industry 4.0 in a structured manner, ensuring that all participants involved share a common perspective and develop a common understanding [1]. The model encompasses three axes: the product life cycle and value stream (distinguishing between types and instances), the hierarchy levels representing different participants within the network, and six interoperability layers covering everything from business processes to communication standards to physical components [1].

Central to this architectural model is the concept of the Digital Twin (DT). A digital twin can be defined as an evolving digital profile of the historical and current

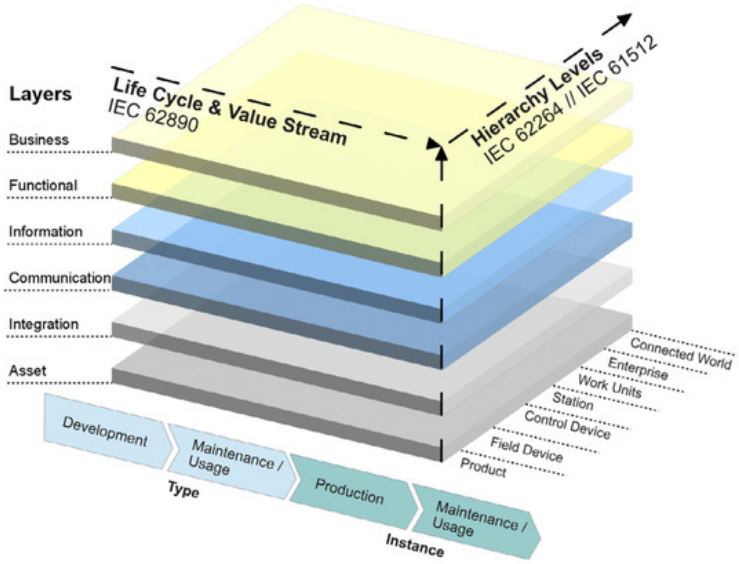


Figure 1.1: Reference Architectural Model Industrie 4.0 (RAMI 4.0) [1]

behavior of a physical object or process that helps optimize business performance [12]. Digital twins allow industry to detect physical issues sooner, predict outcomes more accurately, and build better products [12]. As the adoption of Industry 4.0 advances, digital twins prove invaluable for testing and simulating new parameters and design variants [13].

To fully realize the potential of digital twins in industrial settings, standardization is essential [14]. The AAS has emerged as a key enabler in this regard. The AAS is the logical representation of an asset that enables it to communicate in an Industry 4.0-compliant manner [14]. Together with the asset, the AAS forms the Industry 4.0 Component (I4.0 Component) [14].

The AAS is the standard digital representation of an industrial asset that plays a pivotal role in enabling interoperable communication among I4.0 components across the value chain [14]. The interoperability between I4.0 components depends crucially on the contents of the administration shell, and so it is the main task of the AAS to register and make available the data and functions of all relevant assets in a standardized way throughout their life cycle [15].

However, as manufacturing systems become increasingly complex, autonomous production becomes essential. As [16] note, the increasing complexity caused by global competition, product variety, and individualization demands production sys-

tems that are self-organized and capable of autonomous decision-making [16]. In such environments, it becomes impossible to anticipate and automate all relevant constellations through centralized control [17].

While the AAS provides a robust framework for standardizing asset representation and communication, current implementations predominantly focus on passive data exchange and reactive behavior [18]. Most existing solutions are limited to passive (providing static metadata) or reactive (responding to external events) functionalities, lacking the active behaviors required to support fully autonomous interactions and intelligent decision-making [19]. The Type3AAS, which is characterized by its proactive capabilities, remains largely unexplored in practice, still in its early stages and lacking formal specifications and comprehensive implementation guidelines [18]. Realizing truly autonomous I4.0 components requires not only standardized data models but also decision-making capabilities that allow assets to act proactively based on their goals and beliefs. This gap motivates the integration of agent-based technologies with the AAS concept. MAS offer a promising paradigm for implementing autonomous behavior in distributed systems. In particular, the BDI architecture provides a well-established model for rational agents that reason about their beliefs, desires, and intentions to select appropriate actions [20]. By combining the standardized structure of the AAS with the proactive reasoning capabilities of BDI agents, it becomes possible to realize Type3AAS that can autonomously manage and optimize asset operations.

This thesis proposes an approach to implementing Type3AAS using MAS based on the BDI architecture. The main contributions are:

- A conceptual integration of AAS with BDI-based agents
- Foundational submodels for enabling proactive behavior in AAS
- A proof-of-concept implementation demonstrated through a production resource use case involving drill machines

The remainder of this thesis is structured as follows: Chapter 2 provides a literature review of AAS, MAS, and the BDI architecture. Chapter **??** examines the current state of the art, including existing frameworks and integration approaches.

Chapter 3 defines the system requirements and use case. Chapter 4 presents the conceptual design of the proposed solution. Chapter 5 details the implementation. Chapter 6 evaluates the proof of concept. Finally, Chapter 7 concludes with an outlook on future research directions.

Chapter 2

Fundamentals and State of the Art

2.1 Asset Administration Shells(AAS)

The AAS is one of the central concepts in the principles of Industry 4.0, providing a methodology for designing and implementing the digital representation of manufacturing process components [21]. This section explores the fundamental concepts of the AAS, its structure, and its role in enabling interoperability within Industry 4.0 ecosystems. This section cover the specifications in [3], [4] that are of relevance to this thesis.

2.1.1 Basic concept of AAS

The AAS is a standardized digital representation of an asset, which can be a physical or virtual entity within an industrial environment. It serves as a cornerstone for achieving interoperability and seamless communication of the asset with other components in the Industry 4.0 ecosystem [2]. Together with the asset it represents, the AAS forms the so-called I4.0 Component (I4.0-Component), as illustrated in Figure 2.1. The asset can be any physical or non-physical entity that has value for an organization, including production systems, equipment, machines, components, products, raw materials, business processes, software, documents, and even human personnel [2].

At its core, the AAS is an information model (meta model) that is exchanged

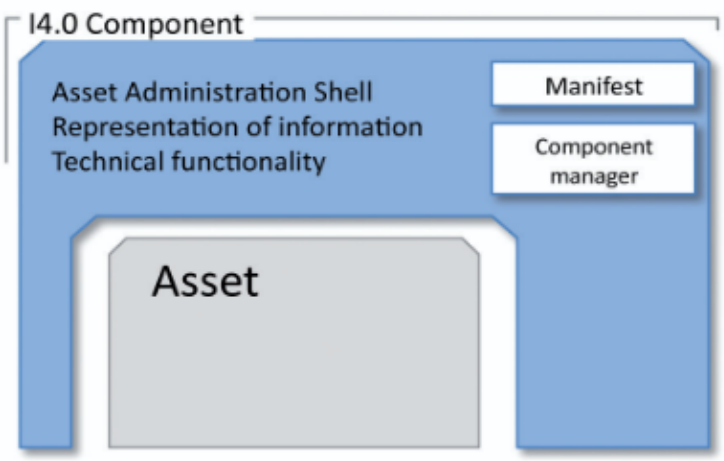


Figure 2.1: The Industry 4.0 Component: Asset and its Administration Shell [2]

between I4.0 components to enable interoperability [3]. This information model could be represented as a *eXtensible Markup Language (XML)*, *JavaScript Object Notation (JSON)* or *Resource Description Framework (RDF)* document [3].

The figure 2.2 illustrates the meta model of the AAS, which defines its structure and key components. According to [3], the AAS has an *Identifiable* attribute, which translate to an element that is referable via a unique identifier.This attribute ensures that each AAS can be distinctly identified within the Industry 4.0 ecosystem which also makes them unambiguous and easy to find. Also it make the AAS into an *AdministrativeInformation* ,which contains information about the versioning , who created the last changes and information about the languages available in case the element contains text for translation [3]. Also each AAS has a *HasDataSpecification* attribute, which means that it can be extended by using data specification templates , those define a named set of additional attributes that an element may or shall have [3]. The AAS also has a *derivedFrom* relation, which allows it to reference another AAS from which it is derived. This relation is useful for establishing hierarchies or relationships between different AASs. The most important attribute of the AAS is the *HasSubmodel* attribute, which indicates that the AAS contains one or more submodels. Submodels are modular components within the AAS that encapsulate specific functionalities and data related to the asset [3]. Finally the AAS has a *AssetInformation* attribute, which provides essential meta data about

the asset itself, including the *AssetKind*, which classify the asset into a *Type AAS* or *Instance AAS* [3]. A *Type AAS* represents a hardware or software element which specifies the common attributes shared by all instances of the type [3]. An *Instance AAS* represents a specific occurrence of an asset, which contains a concrete , clearly identifiable component of the type [3]. The *AssetInformation* attribute also includes the *globalAssetId*, this attribute is required as soon as the AAS is exchanged via partners in the life cycle of the asset. In a first phase of the life cycle the asset might not yet have a global ID but already an internal identifier. The internal identifier would be modelled via [3].

In summary, the AAS provides a standardized framework for representing assets in a digital format, enabling interoperability and seamless communication within Industry 4.0 environments. Its meta model defines the structure and key components of the AAS, laying the foundation for effective asset management and integration in modern industrial systems.

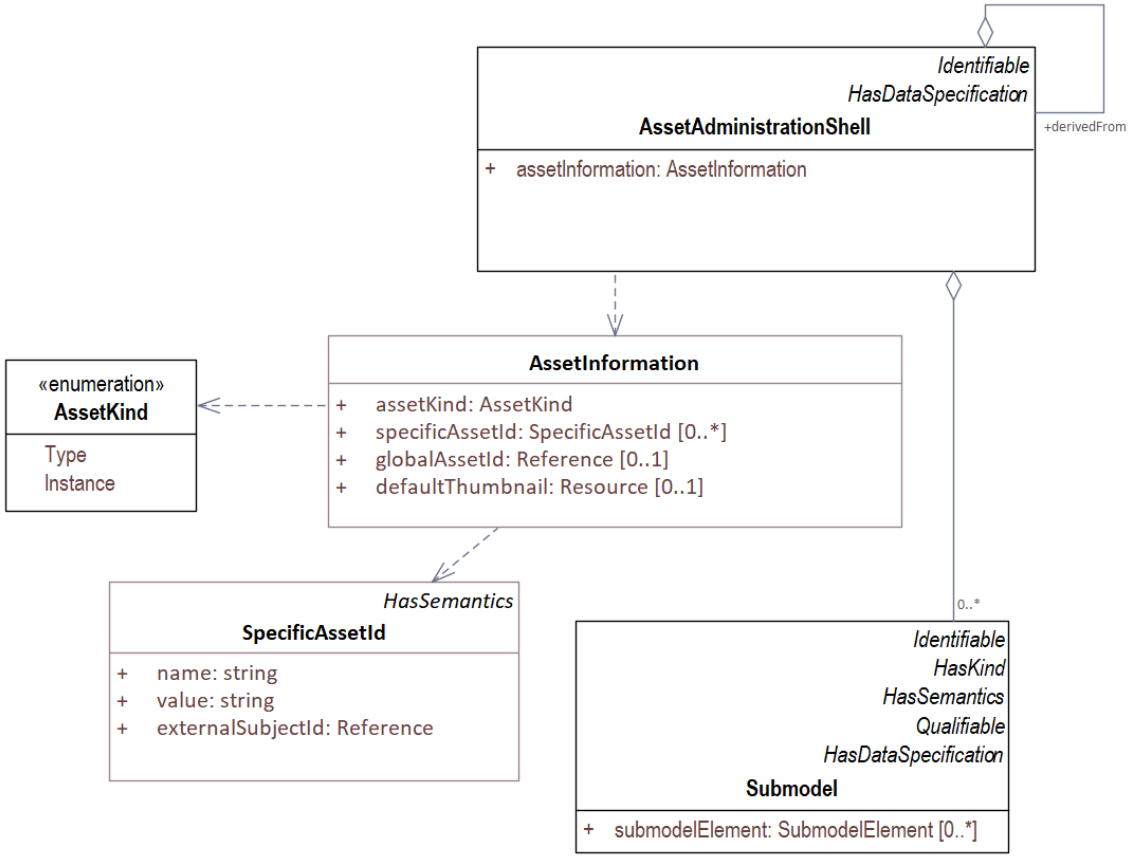


Figure 2.2: Asset Administration Shell Meta Model [3]

2.1.2 Submodels and data types

A submodel is used to structure the digital representation and technical functionality of an Administration Shell into distinguishable parts. Each submodel refers to a well-defined domain or subject matter. Submodels can become standardized and, thus, become submodels templates [3]. In the figure 2.3, the meta model of a submodel is illustrated. Each submodel contains mainly *submodel elements*, which are the building blocks of the submodel structure. *Submodel elements* can represent various types of data and functionalities, including *properties*, *operations*, *events*, and *collections* etc.. [3]. The Submodel also is an *Identifiable* element, which means it has a unique identifier within the context of the AAS [3]. Similar to the AAS, the Submodel also has an *AdministrativeInformation* attribute, which contains versioning and language information [3]. Additionally, the Submodel has a *HasDataSpecification* attribute, allowing it to be extended with data specification templates [3]. This extensibility enables the inclusion of additional attributes and characteristics specific to the submodel's domain [3]. The Submodel can also have a *Qualifier* attribute, which provides additional context or metadata about the submodel element [3]. Finally, the Submodel has a *SemanticId* attribute, which links the submodel to a semantic definition, enabling better understanding and interoperability of the submodel's content [3].

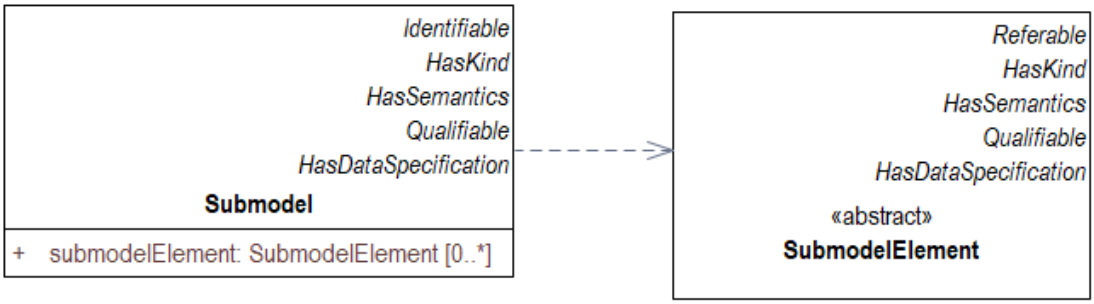


Figure 2.3: Submodel Meta Model [3]

The *SubmodelElement* is the base element for all elements that can be part of a submodel. The *SubmodelElement* can be polymorphic, meaning it takes on different types as shown in figure 2.4.

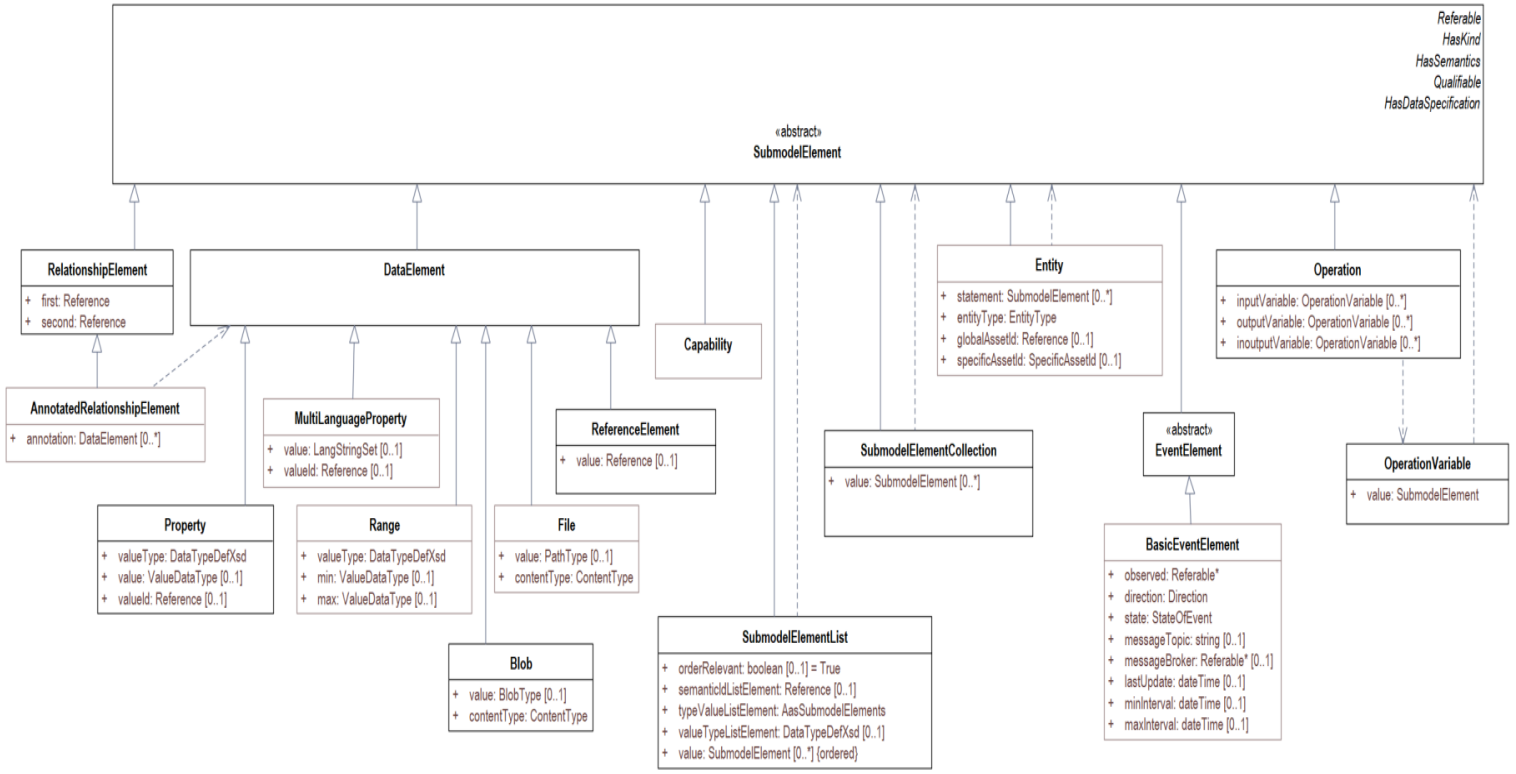


Figure 2.4: Submodel Element Meta Model [3]

In table 2.1, a brief description of the different types of *SubmodelElements* is provided.

Table 2.1: Submodel Element Types and Definitions [3]

Type	Definition	Specification
Property	A data element with a simple value	valueType: DataTypeDefXsd [1..1]
Range	A data element with min and max values	min: ValueDataType [0..1], max: ValueDataType [0..1]
Blob	Binary large object	value: BlobType [0..1], contentType: ContentType
File	Reference to an external file	value: PathType [0..1], contentType: ContentType
ReferenceElement	Reference to another element	value: Reference [0..1]
SubmodelElement-Collection	A collection of submodel elements	value: SubmodelElement [0..*]
SubmodelElement-List	An ordered list of submodel elements	value: SubmodelElement [0..*]
Entity	Represents an entity in a hierarchy	entityType: EntityType [1..1]
RelationshipElement	Defines a relationship between two elements	first: Reference, second: Reference
Annotated-RelationshipElement	Relationship with annotations	annotation: DataElement [0..*]

Continued on next page

Table 2.1 – continued from previous page

Type	Definition	Specification
Capability	Describes a capability of the asset	–
Operation	Executable operation	inputVariable, outputVariable, inoutputVariable
BasicEventElement	Describes an event	observed: Reference [1..1], direction: Direction
MultiLanguage-Property	Property with multi-language support	value: LangStringSet [0..1]
DataElement	Abstract base for data elements	(abstract class)

Along with the different types of *SubmodelElements*, various *data types* are used to define the values and attributes of these elements. These *data types* ensure consistency and standardization in representing information within the AAS and shown in Figure 2.5.

Those *data types* are divided into three main categories:

- **decimalBuildInTypes**: These include basic data types such as *string*, *integer*, *float*, *boolean*, etc., which are used to represent simple values.
- **durationBuildInTypes**: These types are used to represent time durations and intervals, such as *dayTimeDuration* and *yearMonthDuration*.
- **PrimitiveTypes**: This category includes specialized data types like *DateTime*, *Date*, *Time*, *base64Binary*, etc., which provide more specific representations for certain kinds of data.

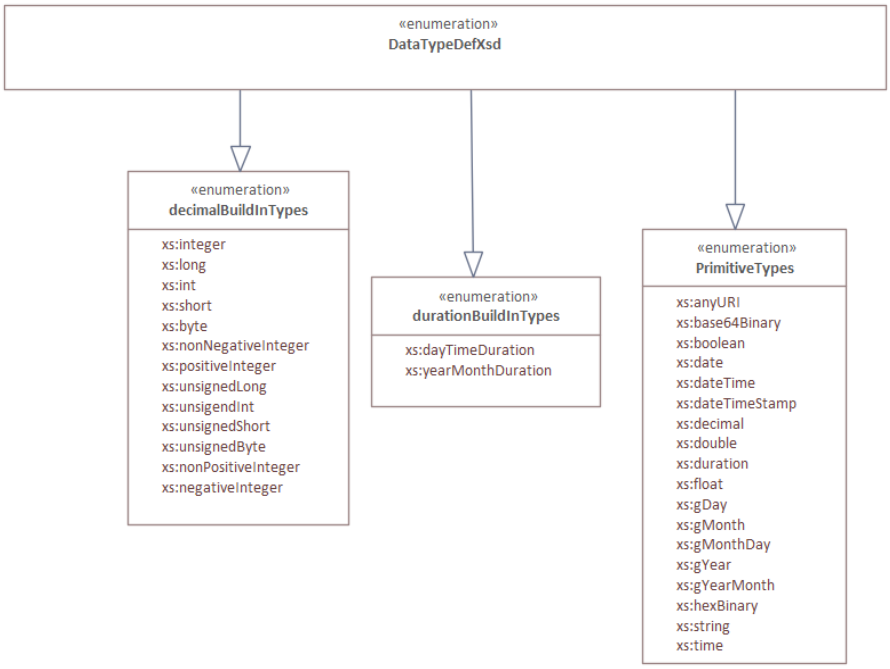


Figure 2.5: Data Types Meta Model [3]

2.1.3 Types of AAS

The AAS can be classified into three types based on the interaction between the AASs [4]. In Figure 2.6, the three types of AAS are illustrated.

The *type1* AAS represents a basic AAS that is covered in previous sections, where those are simply file exchanges between I4.0 components. The *type2* AAS introduces communication capabilities between AASs,where an *API* specification is defined for that purpose [4]. The *type2* AAS allows for the active exchange of information and services between AASs, enabling more dynamic interactions within the Industry 4.0 ecosystem on the contrary of *type1* which is more passive [4]. The *type3* AAS builds on the capabilities of *type2* and the specifications of *type1* where it creates a peer-to-peer network of AASs which allows for a proactive exchange of the system [4]. According to [4], the *type2* AAS has specification that are listed as in the table 2.2.

Table 2.2: Specification of *type2 API* [3]

Specification	Description	Reference
Create, Read, Update, Delete (CRUD) AAS	create, read, update, delete AAS	[3]
CRUD Submodels	create, read, update, delete submodels	[3]
Synchronous (SYNC) Invokers	Invoke methods synchronously	[3]
Asynchronous (ASYNC) Invokers	Invoke methods asynchronously	[3]
Eventing	subscribe and unsubscribe to events	[3]

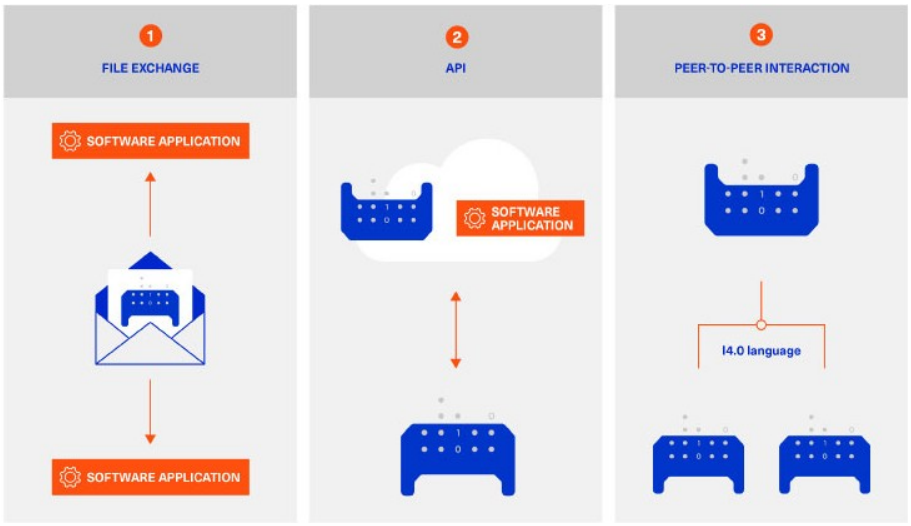


Figure 2.6: Types of Asset Administration Shells [4]

2.1.4 AAS Type3

The *type3* AAS builds upon the capabilities of *type2* AAS by introducing a peer-to-peer network of AASs [5]. The *type3* represents a more advanced and extended form of AAS, acting as a decision-making entity that autonomously interacts with other AASs to exchange information and collaborate (e.g., through negotiation) [5]. The core idea of the Asset Administration Shell AAS concept is that every industrial asset is represented by its own AAS, thereby transforming the asset into an I4.0 component [5]. Because all AASs follow a unified, standardized structure and expose I4.0-compliant interfaces, they enable seamless information provision and exchange across the entire lifecycle of an asset. This standardization facilitates interoperability and real-time data sharing among heterogeneous assets, independent of manufacturer or application domain [22]. In figure 2.7, the concept of *type3* AAS is illustrated. Through their AAS representations, assets are thus envisioned

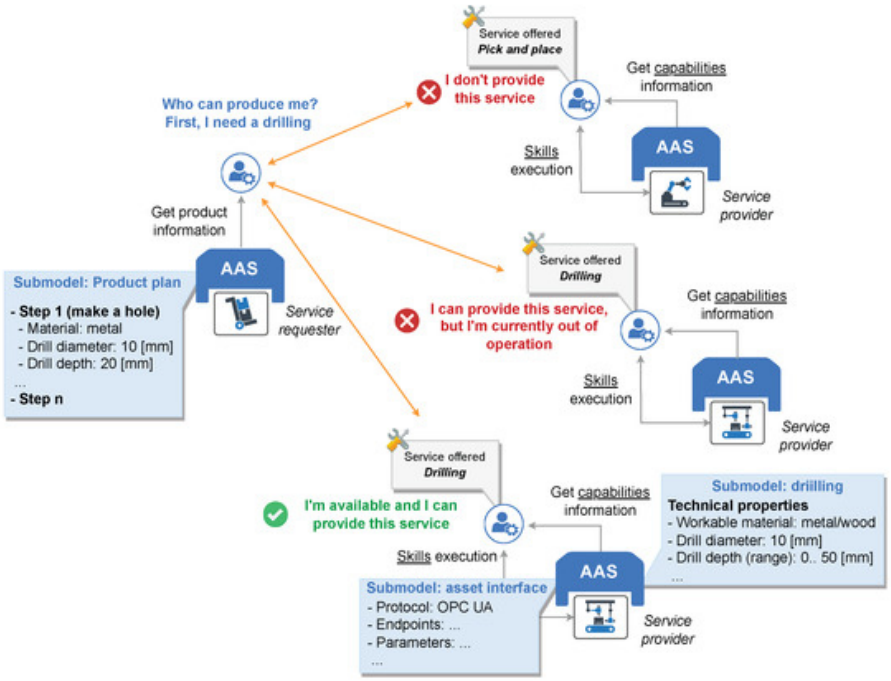


Figure 2.7: Concept of Type3 AAS [5]

to autonomously collaborate, coordinate processes, and contribute to decentralized decision-making within industrial systems [22]. However, such autonomous, intelligent, and cooperative behavior corresponds primarily to AAS *type3*, which unlike

AAS *type1* and *type2* remains comparatively underexplored and currently lacks dedicated specifications and comprehensive implementation guidelines [5]. The *type3* AAS revolves around three main requirements [5] , [22], [23], [24]:

- **Autonomy**: Each AAS should operate independently, makin g decisions and executing actions without human intervention.
- **Intelligence**: AASs should possess cognitive capabilities, enabling them to learn from their environment, adapt to changes, and optimize their performance over time.
- **Collaboration**: AASs should be able to communicate and collaborate with other AASs, forming a network of interconnected agents that work together to achieve common goals.

The *type3* is currently an area of active research and development, with ongoing efforts to define its specifications and explore its potential applications in Industry 4.0 environments [5]. There is to be yet no standardized framework or guidelines for implementing *type3* AAS, making it a promising avenue for future exploration and innovation in the field of industrial automation and digitalization [5].

2.1.5 Summary

This section has introduced the AAS as the standardized digital representation of industrial assets within the Industry 4.0 ecosystem. The key concepts covered include:

- The AAS meta-model, which defines the structure for representing assets through identifiable, self-describing information models that can be exchanged in XML, JSON, or RDF formats.
- Submodels as modular components that encapsulate specific aspects of an asset, including properties, operations, events, and capabilities. Standardized submodel templates enable interoperability across different vendors and application domains.

- The classification of AAS into three types based on their interaction capabilities: Type 1 (file exchange), Type 2 (API-based communication), and Type 3 (peer-to-peer proactive interaction).
- The requirements of Type3AAS—autonomy, proactive behavior, and collaboration—which extend beyond the current specifications and require additional technologies to be realized.

Of particular relevance to this thesis is the Type3AAS, which envisions assets as autonomous entities capable of proactive decision-making and peer-to-peer collaboration. However, as noted, the Type3AAS currently lacks formal specifications and implementation guidelines. The standardized Operation element defined in the AAS meta-model provides a potential interface for executing agent actions, yet this capability remains underutilized in existing implementations. The following sections introduce MAS and the BDI architecture as enabling technologies for realizing the proactive capabilities required by Type3AAS.

2.2 Multi-Agent Systems (MAS)

The concept of autonomous agents and MAS has emerged as a powerful paradigm for designing and implementing complex distributed systems that require autonomous behavior, coordination, and adaptability [25]. This section introduces the fundamental concepts of agents and MAS, establishing the theoretical foundation for understanding their integration with AAS to realize Type3AAS capabilities.

2.2.1 Agents and Their Properties

An agent is commonly defined as "a computer system that is situated in some environment, and that is capable of autonomous action in this environment in order to meet its design objectives" [25]. Unlike conventional software components that execute predefined instructions in response to method calls, agents exhibit a higher degree of autonomy and environmental awareness.

According to [26], an intelligent agent possesses the following essential properties:

- **Autonomy**: Agents operate without direct intervention from humans or other agents. They have control over their own actions and internal state, making independent decisions based on their goals and perceptions.

- **Reactivity**: Agents perceive their environment through sensors or data interfaces and respond in a timely manner to changes that occur in it. This enables agents to adapt to dynamic conditions.

- **Proactivity**: Beyond merely reacting to environmental changes, agents can exhibit goal-directed behavior by taking initiative. They can anticipate future states and act to achieve their objectives without external prompting.

- **Social Ability**: Agents can interact with other agents (and potentially humans) through some form of agent communication language to coordinate activities, negotiate, and cooperate toward shared or individual goals.

These properties distinguish agents from traditional software objects and make them particularly suitable for applications requiring autonomous decision-making and adaptive behavior, such as the proactive capabilities envisioned for Type3AAS.

2.2.2 Multi-Agent Systems

A MAS is a system composed of multiple interacting agents that work together within a shared environment [25]. Each agent in a MAS has its own goals, capabilities, and knowledge, and the collective behavior of the system emerges from the interactions among individual agents.

MAS offer several advantages over centralized systems [27]:

- **Decentralization**: There is no single point of control or failure. Each agent operates autonomously, making the system more robust and resilient to local failures.

- **Scalability**: New agents can be added to the system without requiring fundamental changes to existing agents, allowing the system to grow incrementally.

- **Flexibility**: Agents can dynamically reorganize and adapt their interactions in response to changing requirements or environmental conditions.
- **Modularity**: Each agent encapsulates specific functionality, promoting separation of concerns and facilitating maintenance and evolution of the system.

These characteristics align closely with the requirements of modern manufacturing systems, where flexibility, adaptability, and resilience are essential to cope with increasing product variety, shorter product lifecycles, and unpredictable disruptions [27].

2.2.3 Agent Architectures

Agent architectures define the internal structure of agents and how they process perceptions to generate actions. Three main categories of agent architectures have been identified in the literature [25]:

Reactive Architectures implement a direct mapping from perceptions to actions without maintaining an internal model of the world. These architectures, such as the subsumption architecture proposed by Brooks [28], are computationally efficient and provide fast responses but lack the ability to reason about complex goals or plan ahead.

Deliberative Architectures maintain an explicit symbolic model of the world and use logical reasoning to determine appropriate actions. These architectures can handle complex planning tasks but may be computationally expensive and slow to respond to rapidly changing environments.

Hybrid Architectures combine reactive and deliberative components to balance responsiveness with reasoning capabilities. A typical hybrid architecture consists of multiple layers: a reactive layer for immediate responses, a deliberative layer for planning and reasoning, and a coordination layer that mediates between them. The BDI architecture, discussed in Section 2.3, represents a well-established hybrid approach that provides both reactive capabilities and goal-directed reasoning.

2.2.4 Agent Communication

Communication is fundamental to MAS, enabling agents to share information, coordinate activities, and negotiate agreements. The Foundation for Intelligent Physical Agents (FIPA) has established standards for agent communication that are widely adopted in industrial applications [29].

The FIPA Agent Communication Language (ACL) defines a set of performatives (communicative acts) that specify the intended meaning of messages exchanged between agents. Common performatives include:

- `INFORM`: Used to share information or beliefs with other agents
- `REQUEST`: Used to ask another agent to perform an action
- `PROPOSE`: Used to make a proposal during negotiation
- `ACCEPT-PROPOSAL` / `REJECT-PROPOSAL`: Used to respond to proposals
- `CFP` (Call for Proposals): Used to initiate a negotiation or auction

These standardized performatives enable interoperability between agents developed using different frameworks and facilitate the implementation of coordination protocols such as the Contract Net Protocol [30], which is commonly used for task allocation in manufacturing systems.

2.2.5 Agent Coordination and Cooperation

In MAS, agents must coordinate their activities to achieve common goals and avoid conflicts. Several coordination mechanisms have been developed [25]:

Contract Net Protocol: A market-based mechanism where a manager agent announces a task, receives bids from contractor agents, and awards the contract to the most suitable bidder. This protocol is widely used in manufacturing for dynamic task allocation [30].

Auction-Based Coordination: Agents participate in auctions to acquire resources or tasks. Different auction types (English, Dutch, sealed-bid) offer trade-offs between communication overhead and optimality of allocation.

Negotiation: Agents engage in a dialogue to reach mutually acceptable agreements. Negotiation is essential when agents have conflicting interests but need to cooperate for mutual benefit.

Organizational Structures: Agents can be organized into hierarchies, teams, or coalitions with defined roles and responsibilities. Organizational models such as Moise+ [31] provide formal frameworks for specifying agent organizations.

2.2.6 Industrial Agents

The application of agent technology in industrial settings has given rise to the concept of industrial agents, which are agents specifically designed for manufacturing and production environments [27]. Industrial agents typically operate within a two-layered architecture as illustrated in Figure 2.8:

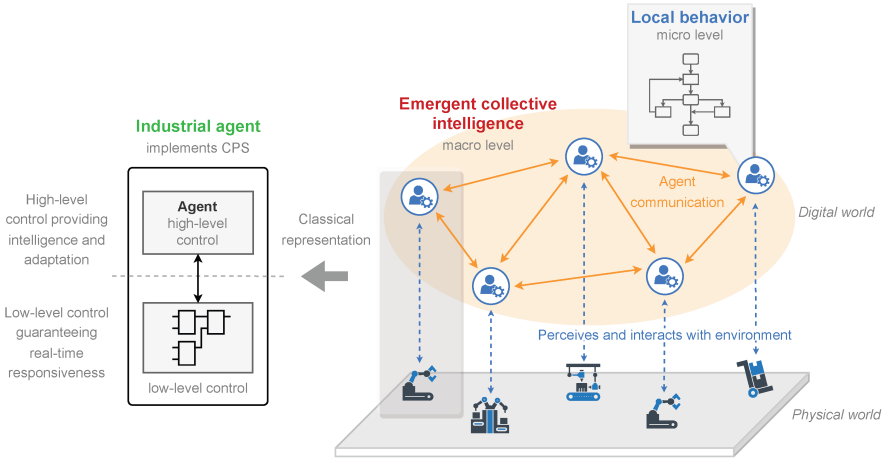


Figure 2.8: Industrial Agent Layers [6]

- **Low-Level Control (LLC)**: This layer handles real-time control of physical assets, including sensor monitoring, actuator control, and safety functions. It is typically implemented using Programmable Logic Controller (PLC)s or industrial PCs with real-time operating systems.
- **High-Level Control (HLC)**: This layer implements the agent logic, including decision-making, coordination, and communication with other agents. It provides intelligence and adaptability while delegating time-critical operations to the LLC layer.

This separation allows agents to leverage the reliability and determinism of industrial control systems while adding intelligent, adaptive behavior through the agent layer. The interface between these layers can be implemented using industrial communication standards such as Open Platform Communications Unified Architecture (OPC UA), which aligns well with the communication capabilities specified for AAS [6].

Industrial agents have been successfully applied in various manufacturing scenarios, including flexible manufacturing systems, supply chain management, and production scheduling [27]. Their ability to operate autonomously, coordinate with peers, and adapt to changing conditions makes them natural candidates for implementing the proactive behavior required by Type3AAS.

2.2.7 Summary

This section has introduced the fundamental concepts of agents and MAS, including agent properties, architectures, communication, and coordination mechanisms. The key characteristics of agents autonomy, reactivity, proactivity, and social ability align directly with the requirements identified for Type3AAS: autonomous operation, intelligent decision-making, and collaborative interaction. Industrial agents provide a proven approach for deploying agent technology in manufacturing environments, with established patterns for integrating intelligent behavior with real-time control systems. The following section examines the BDI architecture, which provides a specific model for implementing rational, goal-directed agents suitable for Type3AAS applications.

2.3 BDI (Belief-Desire-Intention) Approach

The BDI architecture represents one of the most influential and well-established models for implementing rational agents capable of autonomous decision-making [26]. Originally rooted in the philosophical theory of human practical reasoning developed by Bratman [32], the BDI model has been successfully adapted into a computational framework for programming intelligent agents that exhibit goal-directed behavior [20].

2.3.1 Foundations of the BDI Model

The BDI architecture is grounded in three core mental attitudes that guide an agent's reasoning and behavior [32]:

- **Beliefs**: Represent the agent's knowledge about itself, other agents, and its environment. Beliefs are informational states that the agent holds to be true based on its perceptions and prior knowledge. In the context of AAS, beliefs can be derived from submodel data, sensor readings, or information received from other I4.0 components.

- **Desires**: Represent the goals or objectives that the agent would like to achieve. Desires are motivational states that drive the agent's behavior. For a Type3AAS, desires might include optimizing production throughput, minimizing energy consumption, or completing manufacturing orders.

- **Intentions**: Represent the desires that the agent has committed to pursue. Intentions are deliberative states that the agent has decided to act upon, providing a form of commitment that ensures persistence in goal pursuit while allowing for reconsideration when circumstances change significantly.

The interplay between these three attitudes enables agents to reason about their goals, select appropriate plans, and execute actions in a rational manner [20]. This separation of concerns allows for a clear distinction between what the agent knows (beliefs), what it wants to achieve (desires), and what it is currently trying to do (intentions).

2.3.2 The BDI Reasoning Cycle

A BDI agent operates through a continuous reasoning cycle that processes perceptions from the environment and generates actions [33]. The typical reasoning cycle consists of the following steps:

1. **Perception**: The agent perceives changes in its environment and updates its belief base accordingly.
2. **Belief Revision**: Based on new perceptions, the agent revises its beliefs to maintain consistency with the current state of the world.
3. **Option Generation**: The agent generates possible options (potential desires) based on its current beliefs and existing goals.
4. **Deliberation**: The agent deliberates over the available options and selects which goals to commit to as intentions.
5. **Plan Selection**: For each intention, the agent selects an appropriate plan from its plan library that can achieve the intended goal given current beliefs.
6. **Action Execution**: The agent executes the next action in the selected plan, which may affect the environment and trigger new perceptions.

This cycle enables BDI agents to exhibit both reactive behavior (responding to environmental changes) and proactive behavior (pursuing long-term goals), which aligns with the requirements of Type3AAS [26].

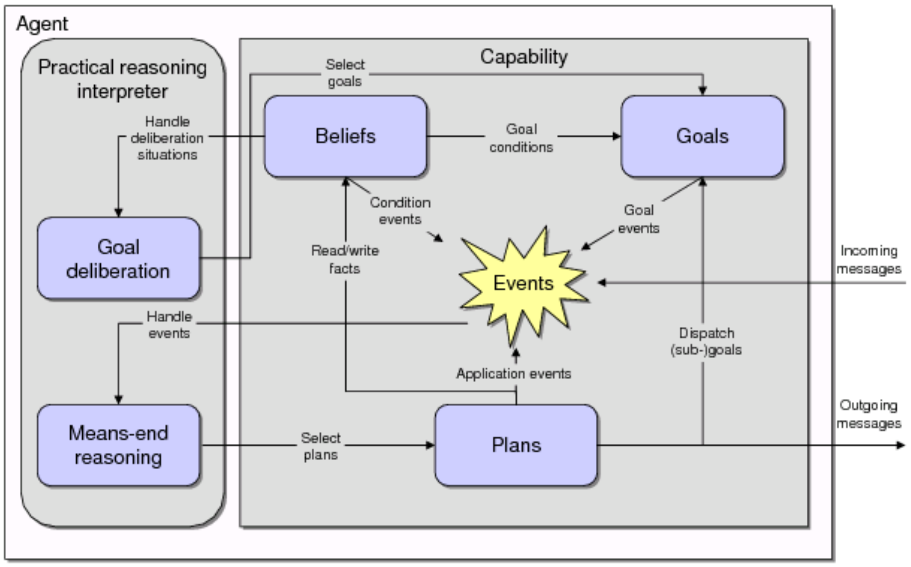


Figure 2.9: BDI Reasoning Cycle [7]

2.3.3 BDI for Industrial and Cyber-Physical Systems

The application of BDI agents in industrial settings has gained significant attention due to their ability to handle complex decision-making in dynamic environments [6]. In the context of CPS and Industry 4.0, BDI agents offer several advantages:

- **Autonomy**: BDI agents can operate independently, making decisions without constant human supervision, which is essential for autonomous manufacturing systems.
- **Adaptability**: The belief revision mechanism allows agents to adapt to changing conditions in real-time, responding to unexpected events such as machine failures or order changes.
- **Explainability**: The explicit representation of beliefs, desires, and intentions provides a transparent model of agent behavior, facilitating debugging, validation, and trust in autonomous systems [34].
- **Goal-Directed Behavior**: The intention-based commitment strategy ensures that agents persistently pursue their goals while maintaining the flexibility to reconsider when necessary.

2.3.4 Summary

The BDI architecture provides a well-founded theoretical and practical framework for implementing rational agents capable of autonomous decision-making. Its clear separation of beliefs, desires, and intentions enables transparent reasoning processes that are essential for industrial applications where explainability and trust are paramount. The availability of mature programming frameworks such as Jason and JaCaMo facilitates the development of BDI-based multi-agent systems. When integrated with the AAS concept, BDI agents can leverage standardized asset information and communication interfaces to realize the proactive, collaborative behavior envisioned for Type3AAS. This integration forms the conceptual basis for the approach proposed in this thesis.

2.4 Software Frameworks and Platforms

The realization of Type3AAS requires not only theoretical foundations but also practical software tools and frameworks. This section reviews the current state of available frameworks for implementing AAS, MAS, and BDI agents, providing the technical context for the implementation decisions made in this thesis.

2.4.1 Asset Administration Shell Frameworks

Several open-source and commercial frameworks have emerged to support the development and deployment of AAS implementations. The most prominent frameworks include:

Eclipse BaSyx is an open-source middleware platform developed under the Eclipse Foundation that provides a comprehensive implementation of the AAS specification [35]. BaSyx offers:

- Complete implementation of the AAS meta-model and API specification
- Support for both Type 1 (file-based) and Type 2 (API-based) AAS
- Registry and discovery services for AAS components

- Integration capabilities with industrial protocols (OPC UA, Message Queuing Telemetry Transport (MQTT), Hypertext Transfer Protocol (HTTP),Representational State Transfer (REST))
- Modular architecture allowing selective deployment of components

BaSyx is widely adopted in research and industry due to its open-source nature and active community support. However, it does not natively support Type3AAS capabilities such as autonomous decision-making or proactive behavior.

Fraunhofer Advanced AAS Tools for Simplified Interoperability (FAAAST) is another open-source framework that focuses on ease of use and rapid prototyping [36]. It provides:

- Lightweight implementation suitable for resource-constrained environments
- Support for multiple serialization formats (JSON, XML, RDF)
- Built-in synchronization with asset data sources
- Extensible architecture for custom functionality

AASX Package Explorer is a desktop application developed by the Industrial Digital Twin Association (IDTA) for creating, viewing, and editing AAS packages [37]. While primarily a development and visualization tool rather than a runtime framework, it serves as an important reference implementation and is commonly used for:

- Creating and validating AAS models
- Importing and exporting AAS packages in standardized formats
- Visualizing submodel structures and relationships
- Testing interoperability between different AAS implementations

Limitations for Type 3 AAS: All current AAS frameworks focus on Type 1 and Type 2 functionality. None provide native support for the autonomous, proactive behavior required by Type3AAS. This gap motivates the integration with agent-based technologies explored in this thesis.

2.4.2 Multi-Agent System Frameworks

Several mature frameworks exist for developing MAS, each with different design philosophies and target applications:

JAVA Agent Development Framework (JADE) is one of the most widely used MAS platforms, providing a FIPA-compliant infrastructure for agent development in Java [38]. Key features include:

- Full compliance with FIPA specifications for agent communication
- Distributed agent platform with support for multiple containers
- Built-in agent management and directory services
- Graphical tools for debugging and monitoring agent behavior
- Behavior-based agent programming model

JADE has been extensively used in industrial applications and research projects. However, its behavior-based programming model does not directly support cognitive agent architectures such as BDI.

Smart Python Agent Development Environment (SPADE) is a modern MAS platform built on Python and Extensible Messaging and Presence Protocol (XMPP) [39]. SPADE offers:

- Asynchronous agent execution using Python's asyncio
- XMPP-based communication providing robust, scalable messaging
- Web-based interface for agent monitoring
- Flexible behavior system with support for cyclic, one-shot, and finite-state behaviors
- Easy integration with Python's extensive ecosystem of libraries

SPADE's use of XMPP provides advantages for industrial deployments, including message persistence, presence detection, and established security mechanisms. Its

Python foundation also facilitates rapid prototyping and integration with data science tools.

JADE eXtension (JADEX) extends JADE with BDI reasoning capabilities, combining JADE's robust infrastructure with cognitive agent programming [40]. JADEX supports:

- BDI agent development using XML-based agent definitions
- Integration with JADE's communication infrastructure
- Goal-oriented programming with automatic plan selection
- Active components model for service-oriented architectures

2.4.3 Framework selection considerations

The selection of appropriate frameworks for implementing Type3AAS depends on several factors:

- **AAS Compliance**: The AAS framework must support the current specification, including submodel operations and standardized APIs.
- **Agent Architecture Support**: For BDI-based implementations, the MAS framework should either natively support BDI or allow integration with BDI reasoning engines.
- **Communication Capabilities**: Industrial deployments require robust, scalable communication mechanisms that can handle asynchronous messaging and potential network disruptions.
- **Integration Flexibility**: The ability to connect AAS and MAS components through well-defined interfaces is essential for realizing bidirectional integration.
- **Development Ecosystem**: Practical considerations such as documentation quality, community support, and available tooling influence development efficiency.

Tables 2.3,2.4 provides a comparative overview of the discussed frameworks.

Table 2.3: Comparison of Asset Administration Shell frameworks

Framework	Language	AAS Types	Communication
Eclipse BaSyx	Java / Python	Type 1, Type 2	HTTP/REST, MQTT, OPC UA
FA³ST Service	Java	Type 1, Type 2	HTTP/REST, OPC UA
AASX Package Explorer	C#	Type 1	File-based (AASX)

Table 2.4: Comparison of Multi-Agent System and BDI frameworks

Framework	Language	BDI Support	Communication
JADE	Java	No (ext.)	FIPA ACL
SPADE	Python	Extension (spade-bdi)	XMPP
JADEX	Java/XML	Yes (native)	FIPA ACL

2.4.4 Summary

This section has reviewed the current landscape of software frameworks for AAS, MAS, and BDI agent development. While mature frameworks exist for each domain individually, no single framework provides integrated support for Type3AAS implementations. AAS frameworks such as Eclipse BaSyx offer comprehensive support for Type 1 and Type 2 functionality but lack autonomous behavior capabilities. MAS frameworks provide the agent infrastructure needed for autonomy and collaboration, with platforms like SPADE offering modern, asynchronous communication. This fragmented landscape necessitates a custom integration approach, which is developed in subsequent chapters of this thesis.

2.5 Integration of AAS with MAS

The preceding sections have established the foundational concepts of AAS as standardized digital representations of industrial assets, MAS as a paradigm for autonomous and collaborative systems, and the BDI architecture as a model for goal-directed agent behavior. This section examines how these concepts can be combined to realize Type3AAS implementations, reviewing existing approaches and identifying the research gaps that this thesis addresses.

As discussed in Section 2.1.4, the requirements of Type3AAS autonomy, proactive behavior, and collaboration align closely with the core properties of agents identified in Section 2.2.1. This alignment has motivated several research efforts to implement Type3AAS using MAS concepts. Table 2.5 summarizes the key approaches found in the literature.

MAS represent a decentralized approach to problem-solving, by distributing tasks and responsibilities among multiple agents autonomously working towards common objectives, on the contrary of traditional monolithic systems where a single entity controls the entire system [27]. There are several attempts to implement *type3* AAS based on MAS concepts. Those attempts are summarized in the table 2.5.

Table 2.5: Summary of AAS Type 3 implementations based on MAS concepts

Reference	Approach	Key Features
Vogel-Heuser et al. (2021) [23]	Approach using AAS as knowledge base for the agents	Demonstrated on a stack-crane system
Sakurada et al. (2022) [41]	Method for implementing AASs using MAS	Decentralized production control, peer-to-peer communication, demonstrated on simulation

Continued on next page

Table 2.5 – continued from previous page

Reference	Approach	Key Features
Siatras et al. (2024) [42]	Agents operate in MAS to solve scheduling problems where the AAS provide production and agent information	Demonstrated on real production line
Sidorenko et al. (2023) [43]	A framework that combines MAS with AAS to enable human-centered manufacturing	Demonstrated on robot with human collaboration
Jungbluth et al. (2022) [44]	Modeling AAS submodels for resource and production agents	Demonstrated on an assembly line

The literature shows consensus that submodels serve as standardized knowledge sources for agents, capturing asset-related information in formats accessible to both humans and machines [5]. Agents leverage these submodels to support decision-making and enrich their knowledge bases. Standardized submodels, such as the Control Component Instance [45], Asset Interfaces Description [46], and Time-Series Data [47], describe the skills offered by an asset, the methods for interfacing with it, and the mechanisms for managing operational data.

Beyond standardized submodels, custom submodels can be defined to address specific application requirements and represent agent-specific information, such as agent behaviors, decision-making strategies, or interaction protocols, thereby enhancing transparency and understanding of agent capabilities [5].

The review further reveals that communication capabilities represent one of the most emphasized and valuable features of agents. This inherent characteristic positions agents as powerful enablers for achieving the decentralized and collaborative interactions particularly through negotiation strategies envisioned in Type3AAS implementations [5].

2.5.1 Critical Analysis of Existing Approaches

The existing approaches to implementing Type3AAS based on MAS concepts, as summarized in Table 2.5, demonstrate promising directions but also reveal significant limitations that motivate further research.

Strengths of Current Approaches

Several positive aspects emerge from the reviewed literature. First, there is a clear consensus on the architectural separation between the AAS as a standardized knowledge repository and the agent as an autonomous decision-making entity [23,41]. This separation allows each component to leverage its respective strengths: the AAS provides interoperability and standardized data access, while agents contribute autonomous behavior and coordination capabilities.

Second, the use of submodels as structured knowledge sources for agents has proven effective across multiple implementations [5, 44]. Standardized submodels such as the Control Component Instance [45] provide a common vocabulary for describing asset capabilities, enabling agents to interpret and reason about heterogeneous assets in a uniform manner.

Third, several approaches have successfully demonstrated decentralized production control through agent negotiation and collaboration [41, 42]. This validates the fundamental premise that MAS can enable the peer-to-peer interactions envisioned for Type3AAS.

Limitations and Open Challenges

Despite these advances, the reviewed approaches exhibit several common limitations:

Unidirectional AAS Usage. Current approaches primarily use the AAS as a passive, read-only data source. Agents retrieve information from submodels to inform their decisions, but the potential of the AAS as an interface for executing actions remains largely unexplored. The AAS specification defines Operations as executable functions that can be invoked synchronously or asynchronously [4], yet existing implementations do not fully leverage this capability for agent action exe-

cution.

Lack of Standardized Submodels for Agent Behavior. While standardized submodels exist for describing asset capabilities (e.g., Control Component Instance [45]) and interfaces (e.g., Asset Interfaces Description [46]), there are no established submodel templates specifically designed to represent agent-related information such as goals, desires, or behavioral parameters. This limits the transparency and interoperability of agent-based Type3AAS implementations.

Validation Predominantly in Simulation. The majority of approaches have been validated only in simulated environments [41] or on simplified laboratory setups [23]. While simulation provides a controlled environment for initial validation, it does not fully capture the complexities, uncertainties, and real-time constraints of actual industrial deployments.

Lack of Standardized Agent-AAS Integration Patterns. Each reviewed approach implements the integration between agents and AAS in a different manner, with no established design patterns or reference architectures. This heterogeneity hinders reusability, comparison, and adoption across different industrial contexts.

Missing Connection Between AAS Operations and Agent Actions. A critical gap exists in how agent intentions are translated into standardized AAS operations. Current implementations typically use proprietary interfaces for agent-asset interaction, bypassing the standardized Operation interface defined in the AAS specification. This undermines the interoperability benefits that the AAS standard aims to provide.

2.5.2 Addressed Research Gap

This thesis addresses the identified limitations by proposing a systematic integration of AAS with BDI-based agents that fully leverages the AAS standard. Specifically, this work contributes:

1. **Bidirectional AAS Integration**: Rather than using the AAS solely as a data source, this thesis proposes a bidirectional integration where agents both read from submodels (to form beliefs) and invoke Operations (to execute in-

tentions). This approach fully utilizes the capabilities specified in the AAS standard, enabling agents to interact with assets through standardized interfaces rather than proprietary solutions.

2. **Foundational Submodels for Proactive Behavior**: This work defines submodel structures specifically designed to support agent-based Type3AAS implementations. These submodels represent goals, capabilities, and behavioral parameters that enable proactive decision-making while maintaining compliance with the AAS meta-model. This contribution addresses the lack of standardized templates for agent-related information.

3. **Proof-of-Concept with Production Resources**: The proposed approach is validated through a concrete use case involving drill machines as production resources. This demonstration shows the feasibility of implementing BDI-based Type3AAS in a realistic production scenario, moving beyond purely simulated environments to a more practical validation context.

By addressing these gaps, this thesis aims to advance the state of the art toward truly interoperable, proactive I4.0 components that leverage the full potential of the AAS standard while benefiting from the autonomous capabilities provided by agent technology.

Chapter 3

Use Cases and System Requirements

The preceding chapter established the theoretical foundations and identified the research gaps that this thesis addresses. Based on these findings, this chapter derives concrete use cases and system requirements for implementing a Type3AAS using BDI-based MAS. Following the systems engineering approach, the use cases are derived from the research questions identified in Section 2.5.1, and the system requirements are subsequently derived from these use cases. Detailed specifications are provided in Appendix 8.1.

3.1 From Research Questions to Use Cases

The critical analysis in Chapter 2 identified three main contributions that this thesis aims to provide: bidirectional AAS integration enabling agents to both read from submodels and invoke Operations, foundational submodels supporting proactive behavior, and a proof-of-concept validation with production resources.

To demonstrate and validate these contributions, a production scenario involving drill machines (in virtual environment) was selected. The *SmartDrillSystem* serves as the proof-of-concept implementation, consisting of multiple AAS-represented drill machines (ResourceAAS) that can be orchestrated by production orders (ProductionAAS). This scenario allows the demonstration of all key Type3AAS capabili-

ties: autonomous behavior, peer-to-peer communication, negotiation, and proactive decision-making.

3.2 System Components

The SmartDrillSystem comprises four main components that work together to realize Type3AAS functionality:

ResourceAAS represents the digital twin of each drill machine, combining the standardized AAS structure with a BDI agent runtime. Each ResourceAAS maintains submodels describing the asset's operational state, capabilities, and interaction parameters, while the embedded agent provides autonomous decision-making and negotiation capabilities.

ProductionAAS manages production orders and coordinates the allocation of tasks to available resources. When a drilling order is received, the ProductionAAS agent queries the registry for available resources, initiates negotiations based on defined criteria (such as time availability), and delegates the production plan to the selected ResourceAAS.

AAS Registry provides a central discovery service where all AAS components register themselves upon initialization. This enables dynamic discovery of available resources without hardcoded dependencies between components.

Event Broker implements an MQTT-based messaging infrastructure that enables asynchronous, event-driven communication between components. This supports real-time status updates and reactive behavior across the distributed system.

3.3 Use Cases

The SmartDrillSystem is specified through three primary use cases that cover the complete lifecycle of the system: Startup, Drilling Order, and Shutdown. Additionally, two included use cases (AAS Registration and Drill Operation) are invoked by the primary use cases to handle specific sub-processes. The relationship between these use cases is illustrated in Figure 3.1.

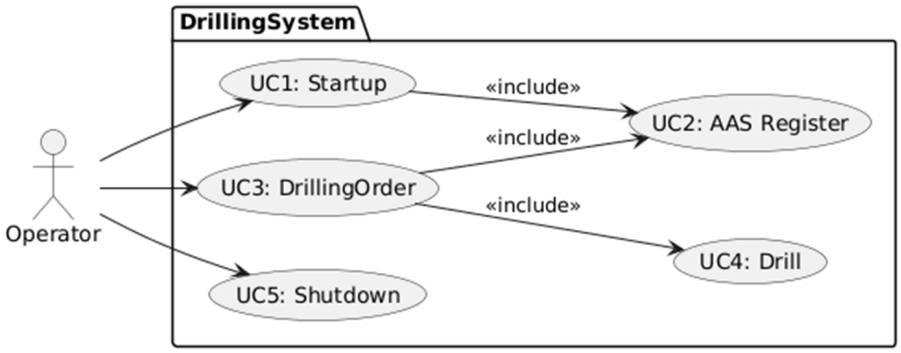


Figure 3.1: Use Case Diagram for the SmartDrillSystem

The detailed use case specifications are provided in Appendix 8.1.1.

3.3.1 UC1: System Startup

The System Startup use case describes the initialization sequence when an operator starts the SmartDrillSystem. During startup, the system initializes all components including the Event Broker, AAS Registry, ResourceAAS instances, and ProductionAAS. The startup process includes the *AAS Registration* (UC2) sub-process, where each ResourceAAS registers itself with the central registry, making it discoverable for subsequent production orders. Upon successful initialization, the system transitions to the ready state and awaits drilling orders. If any component fails to initialize, the system displays appropriate error messages and attempts retry procedures.

3.3.2 UC3: Drilling Order

The Drilling Order use case represents the core production scenario and demonstrates the key Type3AAS capabilities. When an operator submits a drilling order with specified parameters (depth, diameter, material type), the system delegates this order to the ProductionAAS. The ProductionAAS agent then retrieves available resources from the registry and initiates a negotiation process with all registered ResourceAAS agents. Using time availability as the primary negotiation criterion, the ProductionAAS selects the most suitable resource and assigns the production plan.

The selected ResourceAAS then executes the *Drill Operation* (UC4) sub-process,

which handles the actual drilling execution through the bidirectional AAS integration. During this sub-process, the ResourceAAS agent updates its beliefs from submodels, forms intentions, and invokes standardized AAS Operations on the physical asset. Throughout execution, the ResourceAAS publishes status events, and upon completion, the system returns to the ready state for the next order. Alternative flows handle invalid input parameters and drilling failures with appropriate error handling and recovery procedures.

3.3.3 UC5: System Shutdown

The System Shutdown use case ensures graceful termination of the SmartDrillSystem. When an operator initiates shutdown, the system stops all active production processes, completes or cancels pending drilling orders, deregisters all ResourceAAS from the registry, and terminates all components in the proper sequence. This controlled shutdown procedure prevents data loss and ensures system integrity.

3.3.4 Included Use Cases

In addition to the primary use cases described above, two included use cases support the main workflows:

UC2: AAS Registration is included in UC1 (System Startup) and handles the dynamic registration of ResourceAAS instances with the central registry. Each ResourceAAS sends its identification information, capabilities, and communication endpoints to the registry, enabling participation in subsequent negotiation processes.

UC4: Drill Operation is included in UC3 (Drilling Order) and describes the execution of drilling operations by a ResourceAAS. This use case demonstrates the bidirectional integration between the BDI agent and the AAS, where beliefs are derived from submodels and intentions are executed through standardized Operations.

3.4 System Requirements

Based on the use cases defined above, this section summarizes the system requirements organized into functional and non-functional categories. The complete requirements specification with detailed descriptions is provided in Appendix 8.1.

3.4.1 Functional Requirements

The functional requirements are organized into four groups addressing different aspects of the system.

System Specifications (REQ_1 – REQ_8)

The system-level requirements define the overall architecture and fundamental capabilities of the SmartDrillSystem. These requirements specify that the system must comprise ResourceAAS, ProductionAAS, an Event Broker, and an AAS Registry as its core components. The system must provide standardized communication between AAS instances and support event-based messaging through the broker. Furthermore, the requirements mandate that AAS components operate autonomously, possess decision-making capabilities, and collaborate through interdependent processes. These requirements directly address the Type3AAS characteristics of autonomy, intelligence, and collaboration identified in Chapter 2.

ResourceAAS Specifications (REQ_10 – REQ_20)

The ResourceAAS requirements specify the structure and behavior of drill machine representations. Each ResourceAAS must contain an agent interface supporting FIPA ACL communication, a BDI agent runtime, standardized submodels, and an asset interface for physical device communication. The required submodels include an Operational State Submodel (tracking current state and performance KPIs), a Knowledge Submodel (capturing resource constraints and environmental conditions), a Capability Submodel (describing available skills and service parameters), and an Interaction and Scheduling Submodel (defining communication endpoints and collaboration protocols). Additionally, the ResourceAAS must support event

publishing and subscription, and participate in negotiations with ProductionAAS using time availability as the decision criterion.

ProductionAAS Specifications (REQ_21 – REQ_31)

The ProductionAAS requirements define the structure for production order management. Similar to ResourceAAS, each ProductionAAS must contain an agent interface, agent runtime, and specialized submodels. The required submodels include an Identification Submodel (containing order metadata), a Process Plan Submodel (defining tasks, precedences, and required capabilities), a Services Catalog Submodel (listing available machine skills), an Interface and Endpoints Submodel (specifying communication methods), and an Execution State and Tracking Submodel (monitoring current progress and status). The ProductionAAS must coordinate with ResourceAAS through FIPA ACL negotiation protocols.

MAS/BDI Specifications (REQ_32 – REQ_34)

The MAS/BDI requirements specify the integration between agent concepts and the AAS structure. Each ResourceAAS must maintain defined belief sets derived from its submodels, representing the agent's knowledge about itself and its environment. Agent desires (objectives) must be defined for each resource, representing the goals that drive agent behavior. Finally, agent intentions (plans) must be specified, defining the procedures the agent executes to achieve its objectives. These requirements ensure that the BDI architecture is systematically integrated with the AAS meta-model.

3.4.2 Non-Functional Requirements (NFR_1 – NFR_5)

Beyond functional capabilities, the system must satisfy quality requirements essential for industrial deployment. The interoperability requirement mandates compliance with IDTA AAS specifications to ensure compatibility with the broader Industry 4.0 ecosystem. Scalability requirements ensure that new ResourceAAS can be added without modifying existing components. Reliability requirements specify

graceful handling of component failures with appropriate error messages. Maintainability requirements call for modular architecture supporting future extensions. Finally, performance requirements specify that agent negotiations must complete within acceptable time bounds for the proof-of-concept demonstration.

3.5 Requirements Traceability

Table 3.1 provides a traceability matrix linking the thesis contributions to use cases and requirements, ensuring complete coverage of the research objectives.

Table 3.1: Traceability Matrix: Contributions to Requirements

Thesis Contribution	Use Cases	Key Requirements
Bidirectional AAS Integration	UC3, UC4	REQ_5, REQ_14, REQ_32, REQ_34
Foundational Submodels for Proactive Behavior	UC3, UC4	REQ_12, REQ_16–19, REQ_23, REQ_26–30
Proof-of-Concept with Production Resources	UC1–UC5	All requirements

3.6 Summary

This chapter has derived use cases and system requirements for implementing a Type3AAS using BDI-based MAS. Three primary use cases were defined (Startup, Drilling Order, and Shutdown), supported by two included use cases (AAS Registration and Drill Operation) that handle specific sub-processes. From these use cases, 26 functional requirements and 5 non-functional requirements were derived, organized into System Specifications, ResourceAAS Specifications, ProductionAAS Specifications, and MAS/BDI Specifications.

The requirements directly address the research gaps identified in Chapter 2. Bidirectional AAS integration is addressed through requirements for agent-asset communication and the mapping of BDI components to AAS elements. Foundational submodels are specified through detailed submodel requirements for both ResourceAAS and ProductionAAS. The proof-of-concept is enabled through the complete system

specification covering all operational scenarios. The following chapter presents the conceptual design that realizes these requirements.

Chapter 4

Concept

About this chapter

This chapter presents the core concepts underlying the proposed Type3 Asset Administration Shell (AAS) framework and its integration with Multi-Agent Systems (MAS) using the Belief-Desire-Intention (BDI) model. It details the architectural design, key components, and interaction mechanisms that facilitate proactive behavior in industrial assets. The chapter also discusses the theoretical foundations and practical considerations involved in implementing the AAS-MAS integration.

4.1 System overview

About System Overview

This section provides an overview of the proposed Type3 Asset Administration Shell (AAS) integrated with a Multi-Agent System (MAS) using the Belief-Desire-Intention (BDI) approach. It outlines the high-level architecture, key components, and their interactions within the system. The section aims to give readers a comprehensive understanding of how the AAS and MAS work together to achieve enhanced asset management and autonomous decision-making in industrial environments.

4.2 AAS Concept

About AAS Concept

This section delves into the concept of Asset Administration Shells (AAS) as digital representations of physical assets in Industry 4.0. It discusses the structure, components, and functionalities of AAS, with a focus on Type3 AAS and its advanced capabilities. The section also explores how AAS facilitates interoperability, data exchange, and integration with intelligent systems, setting the stage for its combination with Multi-Agent Systems (MAS).

4.3 AAS-MAS Integration Concept

About AAS-MAS Integration Concept

This section explores the concept of integrating Asset Administration Shells (AAS) with Multi-Agent Systems (MAS) using the Belief-Desire-Intention (BDI) approach. It discusses the architectural design, communication mechanisms, and interaction protocols that enable seamless collaboration between AAS and BDI agents. The section highlights the benefits of this integration in enhancing asset management, autonomy, and decision-making capabilities within industrial environments.

4.4 BDI Agent Concept

About BDI Agent Concept

This section delves into the concept of Belief-Desire-Intention (BDI) agents and their role within Multi-Agent Systems (MAS). It discusses the core components of the BDI model, including beliefs, desires, and intentions, and how these elements contribute to autonomous decision-making and goal-oriented behavior in agents. The section also explores the advantages of using BDI agents in industrial applications, particularly in enhancing the functionality and adaptability of Asset Administration Shells (AAS) through intelligent agent integration.

4.5 Proactive AAS Concept

About Proactive AAS Concept

This section introduces the concept of proactive Asset Administration Shells (AAS) and their capabilities in anticipating and responding to dynamic industrial environments. It discusses the design principles, architectural considerations, and interaction mechanisms that enable AAS to exhibit proactive behavior through integration with Multi-Agent Systems (MAS) using the Belief-Desire-Intention (BDI) approach. The section highlights the benefits of proactive AAS in enhancing asset management, autonomy, and decision-making processes.

Chapter 5

Implementation

About this chapter

This chapter details the implementation of the Type3 Asset Administration Shell (AAS) integrated with a Multi-Agent System (MAS) using the Belief-Desire-Intention (BDI) approach. It covers the technical aspects of the AAS development, including the software architecture, key components, and the integration process with the MAS. The chapter also discusses the challenges encountered during implementation and the solutions adopted to address them.

5.1 AAS implementation and repository design pattern

About AAS Implementation and Repository Design Pattern

This section describes the implementation of the Type3 Asset Administration Shell (AAS) and the application of the repository design pattern. It outlines the software architecture, key components, and data management strategies employed in the AAS development. The section also discusses how the repository design pattern facilitates efficient data access, storage, and retrieval within the AAS framework, enhancing its scalability and maintainability.

5.2 Resource Agents implementation

About Resource Agents Implementation

This section details the implementation of Resource Agents within the Multi-Agent System (MAS) integrated with the Type3 Asset Administration Shell (AAS). It discusses the design principles, architectural considerations, and interaction mechanisms that enable Resource Agents to effectively manage and represent physical assets. The section also highlights the role of Resource Agents in facilitating communication and cooperation among agents, as well as their contribution to the overall functionality and autonomy of the integrated AAS-MAS framework.

5.3 Production Agents implementation

About Production Agents Implementation

This section outlines the implementation of Production Agents within the Multi-Agent System (MAS) integrated with the Type3 Asset Administration Shell (AAS). It discusses the design principles, architectural considerations, and interaction mechanisms that enable Production Agents to effectively manage and represent product-related information. The section also highlights the role of Production Agents in facilitating communication and cooperation among agents, as well as their contribution to the overall functionality and autonomy of the integrated AAS-MAS framework.

5.4 MAS communication over XMPP protocols in Asynchronous way

About MAS Communication over XMPP Protocols in Asynchronous Way

This section discusses the implementation of communication mechanisms within the Multi-Agent System (MAS) integrated with the Type3 Asset Administration Shell (AAS) using XMPP protocols in an asynchronous manner . It outlines the architectural design, communication protocols, and interaction patterns that facilitate efficient and reliable message exchange among agents. The section also highlights the benefits of using XMPP for asynchronous communication, including scalability, flexibility, and real-time data transfer, which enhance the overall performance and responsiveness of the integrated AAS-MAS framework.

Chapter 6

Evaluation

About this chapter

This chapter presents a comprehensive evaluation of the Type3 Asset Administration Shell (AAS) integrated with a Multi-Agent System (MAS) using the Belief-Desire-Int ention (BDI) approach. It details the methodologies employed to assess the performance, scalability, and effectiveness of the proposed framework in various industrial scenarios. The chapter also discusses the results obtained from simulations and real-world implementations, highlighting the strengths and potential areas for improvement in the AAS-MAS integration.

Chapter 7

Outlook

About this chapter

This chapter provides an outlook on the future directions and potential advancements in the development and application of Type3 Asset Administration Shells (AAS) integrated with Multi-Agent Systems (MAS) using the Belief-Desire-Intention (BDI) approach. It discusses emerging trends, challenges, and opportunities in the field, as well as suggestions for further research and development to enhance the capabilities and adoption of AAS-MAS frameworks in industrial environments.

Chapter 8

Appendix

8.1 Detailed Use Cases and Requirements

This appendix provides the detailed specifications for use cases and system requirements summarized in Chapter 3.

8.1.1 Use Case Specifications

Table 8.1: Use Case UC1: System Startup

Attribute	Description
Identifier	UC1
Name	System Startup
Description	The SmartDrillSystem initializes all components and prepares for operation.
Actors	Operator
Preconditions	The SmartDrillSystem is shut down.
Trigger	The Operator starts the SmartDrillSystem.

Continued on next page

Table 8.1 – UC1: System Startup (continued)

Attribute	Description
Standard Procedure	1. The Operator starts the SmartDrillSystem (via GUI or CLI). 2. The SmartDrillSystem initializes all system components. 3. The SmartDrillSystem triggers UC2: AAS Register. 4. The SmartDrillSystem displays “ResourceAAS Register Message”. 5. The SmartDrillSystem continues with UC3: DrillingOrder.
Alternative Procedure	2a. If initialization fails → display “SystemComponentsFailMessage”. 2b. Retry initialization. 3a. If registration fails → display “AASRegistryFailureMessage”. 3b. Restart UC1 at step 2.
Postconditions	SmartDrillSystem is in UC3: DrillingOrder OR retrying UC1.
Related Use Cases	UC2: AAS Register

Table 8.2: Use Case UC3: Drilling Order

Attribute	Description
Identifier	UC3
Name	Drilling Order

Continued on next page

Table 8.2 – UC3: Drilling Order (continued)

Attribute	Description
Description	The Operator submits a DrillingOrder to the SmartDrillSystem, which coordinates execution through agent negotiation.
Actors	Operator
Preconditions	The SmartDrillSystem is initialized.
Trigger	The Operator initiates a DrillingOrder.
Standard Procedure	1. The SmartDrillingSystem displays "ReadyForOrder" message. 2. The System prompts the Operator to input DrillingParameters. 3. The Operator inputs DrillingParameters (depth, diameter, material). 4. The Operator submits the DrillingOrder. 5. The System delegates the DrillingOrder to the ProductionAAS. 6. The ProductionAAS retrieves registered ResourceAAS from Registry. 7. The ProductionAAS negotiates with all ResourceAAS. 8. The ProductionAAS sends ProductionPlan to the winning ResourceAAS. 9. The ResourceAAS starts executing the ProductionPlan. 10. The ResourceAAS triggers UC4: Drill. 11. The ResourceAAS publishes DrillingStatusEvent. 12. Upon UC4 completion, ResourceAAS publishes "DoneMessage". 13. The System continues with UC3 at step 1.

Continued on next page

Table 8.2 – UC3: Drilling Order (continued)

Attribute	Description
Alternative Procedure	4a. Invalid DrillingParameters → display "InvalidInputTypeMessage" → return to step 3. 11a. UC4 fails → publish "FailMessage" → display "FailToDrillMessage" → continue at step 3.
Postconditions	SmartDrillSystem is in UC3: DrillingOrder OR UC5: Shutdown.
Related Use Cases	UC2: AAS Register, UC4: Drill

Table 8.3: Use Case UC5: System Shutdown

Attribute	Description
Identifier	UC5
Name	System Shutdown
Description	The SmartDrillSystem performs a graceful shutdown of all components.
Actors	Operator
Preconditions	SmartDrillSystem is in UC1: Startup OR UC3: DrillingOrder.
Trigger	The Operator initiates system shutdown.
Standard Procedure	1. The Operator initiates shutdown of the SmartDrillSystem. 2. The System stops all active ProductionAAS and pending DrillingOrders. 3. The System deregisters all ResourceAAS from the Registry. 4. The System displays "ShutdownMessage".

Continued on next page

Table 8.3 – UC5: System Shutdown (continued)

Attribute	Description
Postconditions	SmartDrillingSystem is shut down.
Related Use Cases	n/A

8.1.2 System Specifications

Table 8.4: System Specification Requirements

ID	Title	Description	Severity	UC
REQ_1	System Components	The System shall contain: 1. One or more ResourceAAS 2. One or more ProductionAAS 3. EventBroker 4. AAS Registry	High	UC1
REQ_2	AAS Communication	The System shall have standardized communication between AASs.	High	UC3
REQ_3	AAS Events	The System shall allow AASs to communicate over Events via an EventBroker.	High	UC3, UC4
REQ_4	AAS Registry	The System shall register all AASs.	High	UC2
REQ_5	Resource-Asset Communication	The System shall create standardized communication between ResourceAAS and PhysicalAsset.	High	UC4
REQ_6	System Autonomy	The System's AAS components shall react autonomously to each other and to external triggers.	High	UC3, UC4
REQ_7	System Intelligence	The System shall contain decision-making capabilities.	High	UC3
REQ_8	System Collaboration	The System's AAS components shall perform interdependent processes.	High	UC3

8.1.3 ResourceAAS Specifications

Table 8.5: ResourceAAS Requirements

ID	Title	Description	Severity	UC
REQ_10	ResourceAAS Components	The ResourceAAS shall contain: 1. Agent Interface (FIPA ACL) 2. Agent Runtime 3. AAS Submodels 4. AAS Asset Interface	High	UC1
REQ_11	ResourceAAS Event	The ResourceAAS shall publish and subscribe to events on system level.	High	UC3, UC4
REQ_12	ResourceAAS Submodels	The ResourceAAS shall contain: 1. Operational State Submodel 2. Knowledge Submodel 3. Capability Submodel 4. Interaction and Scheduling Submodel	Mid	UC4
REQ_13	ResourceAAS Runtime	The ResourceAAS shall have a BDI Agent Runtime.	High	UC3, UC4
REQ_14	Asset Interface	The ResourceAAS shall communicate with the Physical Asset via suited protocol.	High	UC4
REQ_15	Agent Interface	The ResourceAAS shall communicate via FIPA ACL protocol.	High	UC3
REQ_16	Operational State Submodel	Contains: current state, performance KPIs, historical data.	Mid	UC4
REQ_17	Knowledge Submodel	Contains: resource constraints, environmental conditions.	Low	UC4

Continued on next page

Table 8.5 – ResourceAAS Requirements (continued)

ID	Title	Description	Severity	UC
REQ_18	Capability Submodel	Contains: skills, service descriptions.	Mid	UC3
REQ_19	Interaction Submodel	Contains: communication endpoints, collaboration protocols.	Mid	UC3
REQ_20	Negotiation	ResourceAAS shall negotiate using time availability as weight.	High	UC3

8.1.4 ProductionAAS Specifications

Table 8.6: ProductionAAS Requirements

ID	Title	Description	Severity	UC
REQ_21	ProductionAAS Components	The ProductionAAS shall contain: 1. Agent Interface 2. Agent Runtime 3. AAS Submodels	High	UC1
REQ_22	ProductionAAS Events	Shall publish and subscribe to events on system level.	High	UC3
REQ_23	ProductionAAS Submodels	Shall contain: 1. Identification Submodel 2. Process Plan Submodel 3. Services Catalog Submodel 4. Interface and Endpoints Submodel 5. Execution State and Tracking Submodel	High	UC3
REQ_24	Agent Interface	Shall communicate via FIPA ACL protocol.	High	UC3

Continued on next page

Table 8.6 – ProductionAAS Requirements (continued)

ID	Title	Description	Severity	UC
REQ_25	Runtime	Shall have an Agent Runtime.	High	UC3
REQ_26	Identification Submodel	Contains: IDs, version, owner, product context.	Mid	UC3
REQ_27	Process Plan Submodel	Contains: nodes (tasks), edges, entry/exit, capabilities.	Mid	UC3
REQ_28	Services Catalog	Contains: machine skills with callable operations.	Mid	UC3
REQ_29	Interface Submodel	Contains: REST/OPC UA/MQTT endpoints, schemas.	Mid	UC3
REQ_30	Execution State Submodel	Contains: current node, timestamps, step status.	Mid	UC3, UC4
REQ_31	Negotiation	Shall negotiate using time availability as weight.	High	UC3

8.1.5 MAS/BDI Specifications

Table 8.7: MAS/BDI Requirements

ID	Title	Description	Severity	UC
REQ_32	Agent Beliefs	The AAS shall contain a defined Agent belief set for each ResourceAAS.	High	UC4
REQ_33	Agent Desires	The AAS shall contain defined Agent objectives for each ResourceAAS.	High	UC3
REQ_34	Agent Intentions	The AAS shall contain defined Agent plans for each ResourceAAS.	High	UC4

8.1.6 Non-Functional Requirements

Table 8.8: Non-Functional Requirements

ID	Category	Description	Severity
NFR_1	Interoperability	All AAS components shall comply with IDTA AAS specification (Part 1 and Part 2).	High
NFR_2	Scalability	The system shall support adding new ResourceAAS without modifying existing components.	Mid
NFR_3	Reliability	The system shall handle component failures gracefully with appropriate error messages.	High
NFR_4	Maintainability	The system architecture shall follow modular design principles.	Mid
NFR_5	Performance	Agent negotiation shall complete within < 5 seconds for the proof-of-concept.	Mid

Bibliography

[1] M. Lindner, L. Bank, J. Schilp, and M. Weigold, "Digital twins in manufacturing: A rami 4.0 compliant concept," *Sci*, vol. 5, no. 4, p. 40, 2023.

[2] Industrial Digital Twin Association, "Specification of the asset administration shell: Part 1 - metamodel," IDTA, Tech. Rep., 2023, version 3.0.

[3] Platform Industrie 4.0 and Industrial Digital Twin Association, "Details of the asset administration shell - part 1: The exchange of information between partners in the value chain of industrie 4.0," Platform Industrie 4.0 and IDTA, Berlin, Germany, Tech. Rep., 2023, specification Document. [Online]. Available: https://www.plattform-i40.de/IP/Redaktion/EN/Downloads/Publikation/Details_of_the_Asset_Administration_Shell_Part1_V3.pdf

[4] S. Bader, B. Berres, B. Boss, A. G. Gatterburg, M. Hoffmeister, Y. Kogan, A. Köpke, M. Lieske, T. Miny, J. Neidig, A. Orzelski, S. Pollmeier, M. Sauer, D. Schel, T. Schröder, M. Thron, T. Usländer, J. Vialkowitsch, F. Vollmar, J. Wende, and C. Ziesche, "Details of the asset administration shell – part 2: Interoperability at runtime – exchanging information via application programming interfaces, version 1.0," Plattform Industrie 4.0, Berlin, Germany, Tech. Rep. Version 1.0, Nov. 2021, accessed: 2025-12-21. [Online]. Available: https://www.plattform-i40.de/IP/Redaktion/EN/Downloads/Publikation/Details_of_the_Asset_Administration_Shell_Part2_V1.pdf?__blob=publicationFile&v=1

[5] L. Sakurada, F. D. la Prieta, and P. Leitão, "The role of multi-agent systems in realizing asset administration shell type 3," *Future Internet*,

vol. 17, no. 7, p. 270, 2025, accessed: 2025-12-21. [Online]. Available: https://www.mdpi.com/1999-5903/17/7/270

[6] P. Leitão, T. I. Strasser, S. Karnouskos, L. Ribeiro, J. Barbosa, and V. Huang, "Recommendation of best practices for industrial agent systems based on the ieee 2660.1 standard," in *2021 22nd IEEE international conference on industrial technology (ICIT)*, vol. 1. IEEE, 2021, pp. 1157–1162.

[7] A. Pokahr and L. Braubach, *Jadex 0.96x User Guide: Chapter 2. Concepts of the Jadex BDI Reasoning Engine*, Actoron / Distributed Systems Group, University of Hamburg, Hamburg, Germany, Jun. 2007, accessed: 2025-12-21; part of Jadex 0.96x User Guide. [Online]. Available: https://download.actoron.com/docs/releases/jadex-0.96x/userguide/concepts.html

[8] H. Lasi, P. Fettke, H.-G. Kemper, T. Feld, and M. Hoffmann, "Industry 4.0," *Business & information systems engineering*, vol. 6, no. 4, pp. 239–242, 2014.

[9] R. Y. Zhong, X. Xu, E. Klotz, and S. T. Newman, "Intelligent manufacturing in the context of industry 4.0: a review," *Engineering*, vol. 3, no. 5, pp. 616–630, 2017.

[10] S. Vaidya, P. Ambad, and S. Bhosle, "Industry 4.0–a glimpse," *Procedia manufacturing*, vol. 20, pp. 233–238, 2018.

[11] F. Fraile, R. Sanchis, R. Poler, and A. Ortiz, "Reference models for digital manufacturing platforms," *Applied Sciences*, vol. 9, no. 20, p. 4433, 2019.

[12] E. VanDerHorn and S. Mahadevan, "Digital twin: Generalization, characterization and implementation," *Decision support systems*, vol. 145, p. 113524, 2021.

[13] M. Attaran, S. Attaran, and B. G. Celik, "The impact of digital twins on the evolution of intelligent manufacturing and industry 4.0," *Advances in Computational Intelligence*, vol. 3, no. 3, p. 11, 2023.

[14] M. A. Inigo, A. Porto, B. Kremer, A. Perez, F. Larrinaga, and J. Cuenca, "Towards an asset administration shell scenario: a use case for interoperability and standardization in industry 4.0," in *NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 2020, pp. 1–6.

[15] K. Wei, J. Sun, and R. Liu, "A review of asset administration shell," in *2019 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM)*. IEEE, 2019, pp. 1460–1465.

[16] S. Wang, J. Wan, D. Li, and C. Zhang, "Implementing smart factory of industrie 4.0: an outlook," *International journal of distributed sensor networks*, vol. 12, no. 1, p. 3159805, 2016.

[17] O. Antons and J. C. Arlinghaus, "Designing distributed decision-making authorities for smart factories–understanding the role of manufacturing network architecture," *International Journal of Production Research*, vol. 62, no. 1-2, pp. 204–222, 2024.

[18] P. Leitão *et al.*, "The role of multi-agent systems in realizing asset administration shell type 3," *Future Internet*, vol. 17, no. 7, p. 270, 2025.

[19] C. E. Pereira *et al.*, "Asset administration shell tool comparison: A case study with real digital twins used in petrochemical industry," *Sensors*, 2025, pMC11991629.

[20] A. S. Rao, M. P. Georgeff *et al.*, "Bdi agents: From theory to practice." in *Icmas*, vol. 95, 1995, pp. 312–319.

[21] J. Arm, T. Benesl, P. Marcon, Z. Bradac, T. Schröder, A. Belyaev, T. Werner, V. Braun, P. Kamensky, F. Zezulka, C. Diedrich, and P. Dohnal, "Automated design and integration of asset administration shells in components of industry 4.0," *Sensors*, vol. 21, no. 6, p. 2004, 2021.

[22] A. López, O. Casquero, E. Estévez, A. Armentia, D. Orive, and M. Marcos, "An industrial agent-based customizable platform for i4.0 manufacturing

systems," *Computers in Industry*, vol. 146, p. 103859, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S016636152300009X

[23] B. Vogel-Heuser, F. Ocker, and T. Scheuer, "An approach for leveraging digital twins in agent-based production systems," *at-Automatisierungstechnik*, vol. 69, no. 12, pp. 1026–1039, 2021.

[24] L. Monostori, J. Váncza, and S. R. Kumara, "Agent-based systems for manufacturing," *CIRP annals*, vol. 55, no. 2, pp. 697–720, 2006.

[25] M. Wooldridge, *An introduction to multiagent systems*. John wiley & sons, 2009.

[26] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, 1995.

[27] P. Leitao, S. Karnouskos, L. Ribeiro, J. Lee, T. Strasser, and A. W. Colombo, "Smart agents in industrial cyber–physical systems," *Proceedings of the IEEE*, vol. 104, no. 5, pp. 1086–1101, 2016.

[28] R. A. Brooks, "A robust layered control system for a mobile robot," *IEEE Journal on Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986.

[29] Foundation for Intelligent Physical Agents, "Fipa acl message structure specification," FIPA, Tech. Rep. SC00061G, 2002. [Online]. Available: http://www.fipa.org/specs/fipa00061/SC00061G.html

[30] R. G. Smith, "The contract net protocol: High-level communication and control in a distributed problem solver," *IEEE Transactions on Computers*, vol. C-29, no. 12, pp. 1104–1113, 1980.

[31] J. F. Hübner, J. S. Sichman, and O. Boissier, "Developing organised multiagent systems using the moise+ model: Programming issues at the system and agent levels," *International Journal of Agent-Oriented Software Engineering*, vol. 1, no. 3/4, pp. 370–395, 2007.

[32] M. E. Bratman, *Intention, Plans, and Practical Reason.* Cambridge, MA: Harvard University Press, 1987.

[33] R. H. Bordini, J. F. Hübner, and M. Wooldridge, *Programming Multi-Agent Systems in AgentSpeak Using Jason*, ser. Wiley Series in Agent Technology. Wiley, 2007.

[34] E. Yan, S. Burattini, J. F. Hübner, and A. Ricci, "A multi-level explainability framework for engineering and understanding bdi agents," *Autonomous Agents and Multi-Agent Systems*, vol. 39, no. 1, p. 9, 2025.

[35] Eclipse Foundation, "Eclipse basyx: Open source platform for industry 4.0," 2023, accessed: 2025-12-21. [Online]. Available: https://www.eclipse.org/basyx/

[36] Fraunhofer IOSB, "Fa³st service: Fraunhofer advanced asset administration shell tools," 2023, accessed: 2025-12-21. [Online]. Available: https://github.com/FraunhoferIOSB/FAAAST-Service

[37] Industrial Digital Twin Association, "Aasx package explorer," 2023, accessed: 2025-12-21. [Online]. Available: https://github.com/admin-shell-io/aasx-package-explorer

[38] Telecom Italia, "Jade: Java agent development framework," 2023, accessed: 2025-12-21. [Online]. Available: https://jade.tilab.com/

[39] M. E. Gregori *et al.*, "Spade: Smart python agent development environment," 2023, accessed: 2025-12-21. [Online]. Available: https://spade-mas.readthedocs.io/

[40] Actoron GmbH, "Jadex active components," 2023, accessed: 2025-12-21. [Online]. Available: https://www.2activecomponents.2org/

[41] L. Sakurada, P. Leitao, and F. De La Prieta, "Engineering a multi-agent systems approach for realizing collaborative asset administration shells," in *2022 IEEE International Conference on Industrial Technology (ICIT).* IEEE, 2022, pp. 1–6.

[42] V. Siatras, E. Bakopoulos, P. Mavrothalassitis, N. Nikolakis, and K. Alexopoulos, "Production scheduling based on a multi-agent system and digital twin: a bicycle industry case," *Information*, vol. 15, no. 6, p. 337, 2024.

[43] A. Sidorenko, W. Motsch, M. van Bekkum, N. Nikolakis, K. Alexopoulos, and A. Wagner, "The mas4ai framework for human-centered agile and smart manufacturing," *Frontiers in Artificial Intelligence*, vol. 6, p. 1241522, 2023.

[44] S. Jungbluth, J. Hermann, W. Motsch, M. Pourjafarian, A. Sidorenko, M. Volkmann, K. Zoltner, C. Plociennik, and M. Ruskowski, "Dynamic replanning using multi-agent systems and asset administration shells," in *2022 IEEE 27th International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, 2022, pp. 1–8.

[45] Industrial Digital Twin Association, "Submodel template for control component instance," Industrial Digital Twin Association (IDTA), Frankfurt am Main, Germany, Tech. Rep. IDTA 02016-1-0, April 2023, version 1.0. [Online]. Available: https://industrialdigitaltwin.org/wp-content/uploads/2023/04/IDTA-02016-1-0-_Submodel_ControlComponentInstance.pdf

[46] I. D. T. Association *et al.*, "Idta 02017-1-0 asset interfaces description," *Industrial Digital Twin Association*, 2024.

[47] ——, "Idta 02008-1-1 time series data," *Industrial Digital Twin Association: Frankfurt am Main, Germany*, 2023.